



PROJECT BERICHT

**11IT2B
- 2026 -**



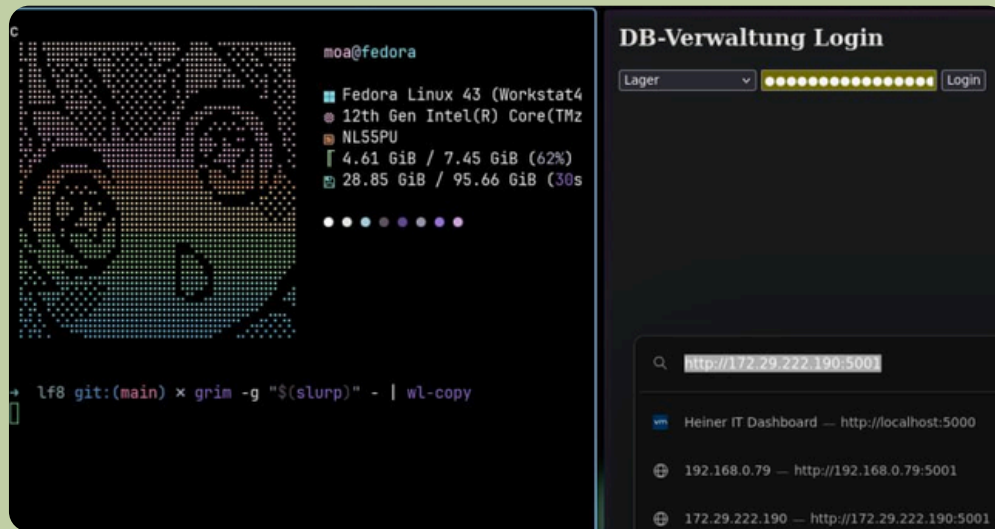
Gruppe Mitglieder

Amirhossein Gozashti

Reza Abutalebi

Elias Könemann

EINLEITUNG



Worum geht es im Projekt?

Im Rahmen dieses Projekts wurde eine webbasierte Anwendung entwickelt, die es Benutzern ermöglicht, auf Daten aus einer MySQL-Datenbank zuzugreifen und diese zu verwalten.

Die Anwendung besteht aus mehreren Komponenten. Auf einem Schulserver läuft ein Python-Programm, das die Verbindung zur Datenbank herstellt und die gespeicherten Informationen verarbeitet. Die Daten werden aus einer MySQL-Datenbank ausgelesen und für die weitere Nutzung bereitgestellt.

Zusätzlich wurde eine benutzerfreundliche Website entwickelt. Über diese Website können sich Benutzer anmelden und neue Datensätze eingeben. Die eingegebenen Informationen werden automatisch in der MySQL-Datenbank gespeichert. Dadurch wird die direkte Arbeit mit der Datenbank für den Benutzer vereinfacht.

Das Ziel des Projekts war es, eine funktionierende Verbindung zwischen einer Webanwendung, einem Python-Backend und einer MySQL-Datenbank aufzubauen. Dabei wurden Kenntnisse in den Bereichen Webentwicklung, Datenbanken, Benutzerverwaltung und Programmierung mit Python angewendet.

```
(.venv) → lf8 git:(main) python 01_ConnectionTest.py
Datenbankverbindung erfolgreich!
(.venv) → lf8 git:(main) sudo nano 01_ConnectionTest.py
[sudo] password for moa:
(.venv) → lf8 git:(main) × python 01_ConnectionTest.py
Error connecting to MariaDB Platform: Can't connect to server on '10.145.240.128' (115)
```

1.1 Verbindung zur Datenbank herstellen

CONNECTION TEST

Dieser Programmabschnitt stellt die Verbindung zwischen der Python-Anwendung und der MariaDB-/MySQL-Datenbank her. Zunächst werden die benötigten Bibliotheken mariadb und sys importiert. Die Bibliothek mariadb wird für die Kommunikation mit der Datenbank verwendet, während sys genutzt wird, um das Programm bei schwerwiegenden Fehlern kontrolliert zu beenden.

Anschließend wird innerhalb eines try-except-Blocks versucht, eine Verbindung zur Datenbank aufzubauen. Hierfür werden die Verbindungsdaten wie Server-IP-Adresse, Benutzername, Passwort und Datenbankname angegeben. Falls die Verbindung erfolgreich hergestellt wird, steht das Verbindungsobjekt für weitere Datenbankabfragen zur Verfügung.

Tritt während des Verbindungsaufbaus ein Fehler auf, wird eine entsprechende Fehlermeldung ausgegeben und das Programm beendet.

Zusätzlich wurde die Funktion testConnection() implementiert. Diese Funktion überprüft die bestehende Datenbankverbindung durch die Ausführung einer einfachen SQL-Abfrage (SELECT 1). Liefert die Datenbank ein Ergebnis zurück, wird die Verbindung als erfolgreich eingestuft. Andernfalls wird eine Fehlermeldung ausgegeben.

Durch den Aufruf der Funktion am Ende des Programms wird sichergestellt, dass die Datenbankverbindung vor der weiteren Verarbeitung der Daten korrekt funktioniert.

CODE

```
import mariadb
import sys

try:
    db = mariadb.connect(
        host="10.145.240.127", # Server-IP-Adresse
        user="root",          # Benutzer
        password="123",       # Passwort
        database="Heiner_IT"  # Datenbankname
    )
except mariadb.Error as e:
    print(f"Error connecting to MariaDB Platform: {e}")
    sys.exit(1)

# Die Methode prüft ob eine erfolgreiche Verbindung zur Datenbank
# hergestellt werden kann.
# Ausgabe: gibt "successful" oder "failed" als print-Ausgabe zurück
def testConnection dbc: # dbc steht für die Datenbankverbindung die
    überprüft werden soll
    """
    Testet die Datenbankverbindung durch eine einfache Abfrage

    Parameter:
        dbc: Datenbankverbindungsobjekt

    Rückgabe:
        Gibt "successful" oder "failed" aus
    """
    try:
        cursor = dbc.cursor() # Erstelle einen Cursor zur Ausführung von
        SQL-Befehlen
        cursor.execute("SELECT 1") # Einfache Test-Abfrage
        result = cursor.fetchone() # Hole das Ergebnis
        cursor.close() # Schließe den Cursor

        if result:
            print("Datenbankverbindung erfolgreich!")
            return "successful"
        else:
            print("Datenbankverbindung fehlgeschlagen!")
            return "failed"
    except mariadb.Error as e:
        print(f"✗ Fehler: {e}")
        return "failed"

# Aufruf der Methode zum Testen der Datenbankverbindung
testConnection(db)
```

1.2 SQL-Abfrage ausführen und Ergebnisse ausgeben

Nachdem die Verbindung zur Datenbank erfolgreich hergestellt wurde, dient dieser Programmabschnitt dazu, Daten aus der Datenbank abzurufen und übersichtlich auszugeben. Zunächst wird die SQL-Anweisung `SELECT * FROM personal` definiert. Diese Abfrage liest alle Datensätze aus der Tabelle `personal` aus. Die Tabelle enthält die Informationen der gespeicherten Mitarbeiter.

```
(.venv) → lf8 git:(main) × python printSQL.py
```

PersonalNr	Durchwahl	Nachname	Bemerkungen	Vorname	Vorgesetzt	Position	Gehalt	Anrede	Geburtsdatum
1	5467	Huber		Clara	100	Sekretariat	2800.00	Frau	1972-01-01
2	3457	Rosnick		Peter	8	Vertriebsmitarbeiter	3920.00	Herr	1968-03-15
3	3355	Weber		Rolf	8	Vertriebsmitarbeiter	3584.00	Herr	1990-05-20
4	5176	Schaller		Werner	100	Lagerleiterin	3752.00	Frau	1975-11-08
5	3453	Kowalski		Pawel	103	PC Konfiguration	3472.00	Herr	1985-07-03
6	428	Yamamoto		Hashiro	100	Netzwerke Leiter	4233.60	Herr	1992-09-12
7	465	Schmidt		Robert	6	Netzwerke	3248.00	Herr	1978-04-25
8	2344	Can		Ayşe	100	Vertriebskoordinatorin	4452.00	Frau	1980-02-10
9	452	Benetzkı		Anne	8	Vertriebsmitarbeiterin	3690.40	Frau	1988-06-18
11	434	Mayer		Charlotte	100	Leiterin Controlling	4536.00	Frau	1995-10-05
12		Mayer		Carl		Vertriebsmanager		Herr	1970-08-22

Die Funktion `testprint()` übernimmt anschließend die Ausführung der SQL-Abfrage. Hierfür wird zunächst ein Cursor erstellt, der als Schnittstelle zwischen Python und der Datenbank dient. Über den Cursor wird die SQL-Anweisung an die Datenbank gesendet und ausgeführt.

Nach erfolgreicher Ausführung werden die Spaltennamen der Tabelle automatisch aus den Metadaten der Abfrage ausgelesen. Dadurch können die Überschriften dynamisch erzeugt werden, ohne dass sie im Code fest hinterlegt werden müssen.

Anschließend werden alle Datensätze mit `fetchall()` abgerufen und in einer übersichtlichen Tabellenstruktur auf der Konsole ausgegeben. Jede Zeile entspricht dabei einem Datensatz aus der Tabelle `personal`.

Zum Abschluss wird der Cursor geschlossen, um die verwendeten Ressourcen wieder freizugeben. Sollte während der Ausführung ein Fehler auftreten, wird eine entsprechende Fehlermeldung ausgegeben. Dieser Programmteil ermöglicht es, die in der Datenbank gespeicherten Mitarbeiterdaten auszulesen und für Kontroll- oder Analysezwecke übersichtlich darzustellen.

```
import mariadb
import sys

try:
    db = mariadb.connect(
        host="10.145.240.127",
        user="root",
        password="123",
        database="Heiner_IT"
    )
except mariadb.Error as e:
    print(f"Fehler: {e}")
    sys.exit(1)

sql_Anweisung = "SELECT * FROM personal" # Alle Mitarbeiter aus der personal-Tabelle

def testprint(dbc, sql_Anweisung_str):
    """
    Führt eine SQL-Abfrage aus und gibt die Ergebnisse formatiert aus

    Parameter:
        dbc: Datenbankverbindungsobjekt
        sql_Anweisung_str: Die auszuführende SQL-Anweisung
    """
    try:
        cursor = dbc.cursor() # Erstelle Cursor für SQL-Befehle
        cursor.execute(sql_Anweisung_str) # Führe SQL-Befehl aus

        # Hole die Spaltennamen aus den Metadaten
        column_names = [desc[0] for desc in cursor.description]
        print("\n" * 100)
        print(" | ".join(f"{col:20}" for col in column_names))
        print("=" * 100)

        for row in cursor.fetchall():
            print(" | ".join(f"{str(val):20}" for val in row))

        print("\n" * 100 + "\n")
        cursor.close() # Schließe den Cursor

    except mariadb.Error as e:
        print(f"Fehler bei der Abfrage: {e}")
    testprint(db, sql_Anweisung)
```

CODE

```
(.venv) → lf8 git:(main) × ls
01_ConnectionTest.py 2.1.py 2.2.py CSVExport.py CSVtoXML.py GUITabelle.py login_log.txt main.py printSQL.py README.MD v7dashboard.py XMLtoHTML.py
(.venv) → lf8 git:(main) × python CSVExport.py
✓ CSV-Export erfolgreich: lagerbestand.csv
(.venv) → lf8 git:(main) × ls
01_ConnectionTest.py 2.1.py 2.2.py CSVExport.py CSVtoXML.py GUITabelle.py lagerbestand.csv login_log.txt main.py printSQL.py README.MD v7dashboard.py XMLtoHTML.py
(.venv) → lf8 git:(main) × grim -g "§(slurp)" - | wl-copy
```

1.3 EXPORT VON DATENBANKDATEN IN EINE CSV-DATEI

Dieser Programmabschnitt ermöglicht den Export von Daten aus einer Datenbanktabelle in eine CSV-Datei. Dadurch können die gespeicherten Informationen außerhalb der Datenbank weiterverarbeitet, analysiert oder beispielsweise mit Tabellenkalkulationsprogrammen wie Microsoft Excel geöffnet werden.

Nach dem Aufbau der Datenbankverbindung wird die Funktion `tabelle_to_csv()` aufgerufen. Die Funktion erhält als Parameter den Namen der zu exportierenden Tabelle, das Datenbankverbindungsobjekt sowie den Namen der Ausgabedatei.

Innerhalb der Funktion wird zunächst ein Cursor erstellt und eine SQL-Abfrage ausgeführt, welche alle Datensätze der angegebenen Tabelle ausliest. Anschließend werden die Spaltennamen automatisch aus den Metadaten der Datenbankabfrage übernommen. Dadurch werden die Überschriften der CSV-Datei automatisch erzeugt.

Mit `fetchall()` werden alle Datensätze der Tabelle abgerufen und in einer Liste gespeichert. Danach wird eine neue CSV-Datei erstellt. Die Bibliothek `csv` übernimmt dabei das Schreiben der Daten in das gewünschte Format. Als Trennzeichen wird ein Semikolon verwendet, da dieses Format häufig im deutschsprachigen Raum genutzt wird.

Zuerst werden die Spaltenüberschriften in die CSV-Datei geschrieben. Anschließend werden alle Datensätze aus der Datenbank zeilenweise exportiert. Nach erfolgreichem Abschluss erscheint eine Bestätigungsmeldung auf der Konsole.

Im vorliegenden Beispiel wird die Tabelle `artikel` exportiert und als Datei `lagerbestand.csv` gespeichert. Dadurch steht der aktuelle Lagerbestand außerhalb der Datenbank zur Verfügung und kann für Auswertungen oder Dokumentationen genutzt werden.

CODE

```
##### CSV-Export aus Datenbank #####
```

```
import mariadb
import csv
import sys
```

```
# Verbindung zur Datenbank
try:
```

```
    db = mariadb.connect(
        host="10.145.240.127",
        user="root",
        password="123",
        database="Heiner_IT"
    )
```

```
except mariadb.Error as e:
    print(f"Fehler: {e}")
    sys.exit(1)
```

```
# SQL-Abfrage für den Lagerbestand
sql_bestand = "SELECT * FROM artikel" # Alle Artikel aus der artikel-Tabelle
```

```
# Funktion zum Exportieren einer Tabelle in CSV
def tabelle_to_csv(tabelle_name, dbc, output_file="export.csv"):
```

```
    """
    Exportiert eine Datenbanktabelle in eine CSV-Datei
```

```
    Parameter:
```

```
    tabelle_name: Name der zu exportierenden Tabelle
    dbc: Datenbankverbindungsobjekt
    output_file: Name der Ausgabedatei (Standard: export.csv)
```

```
    """
```

```
    try:
        cursor = dbc.cursor() # Erstelle Cursor
        cursor.execute(f"SELECT * FROM {tabelle_name}") # Wähle alle Daten aus
```

```
    # Hole Spaltennamen aus Metadaten
    column_names = [desc[0] for desc in cursor.description]
```

```
    # Hole alle Daten
    rows = cursor.fetchall()
```

```
    # Schreibe in CSV-Datei
    with open(output_file, 'w', newline="", encoding='utf-8') as csvfile:
        writer = csv.writer(csvfile, delimiter=";") # Verwende Semikolon als
    Trennzeichen
        writer.writerow(column_names) # Schreibe Spaltenköpfe
        writer.writerows(rows) # Schreibe alle Datenzeilen
```

```
    print(f"✓ CSV-Export erfolgreich: {output_file}")
    cursor.close()
```

```
    except mariadb.Error as e:
        print(f"✗ Fehler beim CSV-Export: {e}")
```

```
# Aufruf für den Artikel-Lagerbestand
tabelle_to_csv("artikel", db, "lagerbestand.csv")
```

1.4 Grafische Darstellung der Daten mit Tkinter

```
(.venv) + lf8 git:(main) x p + lf8 git:(main) x grim -g "$
Python GUI_Tabelle.py
(slurp)" - | wl-copy
selection cancelled
invalid geometry
+ lf8 git:(main) x grim -g "$
(slurp)" - | wl-copy
selection cancelled
invalid geometry
+ lf8 git:(main) x grim -g "$
(slurp)" - | wl-copy
```

ArtikelNr	Artikelname	LieferantenNr	KategorieNr	Liefereinheit	Einzelpreis	Lagerbestand	BestellteEinheiten	Mindestbestand	Ausla
1	Scan Disk Cruiser 8	1	1	Blister mit 2 Stück	46.00	39	0	10	0
2	Scan Disk Cruiser 8	1	1	Blister mit 4 Stück	53.00	17	40	25	0
3	Midi Tower schw	1	2	2 Stück	68.00	13	70	25	0
4	Midi Tower rot Pre	2	2	5 Stück	217.00	53	0	0	0
5	Sharkoon VG4-Sch	2	1	1 Stück	38.00	0	0	0	1
6	Corsair Graphite S	2	2	2 Stück	257.00	18	0	10	0
7	BenQ 21 Zoll	3	7	1 Stück	129.00	15	0	10	0
8	Thermaltake A340	3	2	1 Stück	48.00	6	0	0	0
9	Maus Basic Kabel	4	6	5 Stück	24.00	29	0	10	1
10	Ikura Tablet	4	8	1 Stück	69.00	31	0	0	0
11	HDMI Kabel 1,5 m	5	4	Karton x 5 Einheits	21.00	22	30	30	0
12	HDMI Kabel 2,5 m	5	4	Karton x 5 Einheits	38.00	86	0	0	0
13	Samsung Galaxy T	6	8	1 Stück	160.00	24	0	5	0
14	Dell Touch Monito	6	7	1 Stück	232.00	35	0	0	0
15	Viper Raidmax MY	6	2	1 Stück	63.00	39	0	5	0
16	Seagate 250 GB	7	3	1 Stück	70.00	29	0	10	0

Dieser Programmabschnitt dient zur Entwicklung einer grafischen Benutzeroberfläche (GUI) mit der Python-Bibliothek Tkinter. Ziel ist es, die Daten aus der Datenbank nicht nur in der Konsole, sondern in einer benutzerfreundlichen Tabellenansicht darzustellen.

Zunächst wird eine Verbindung zur Datenbank hergestellt. Anschließend übernimmt die Funktion `read_from_database()` das Auslesen der gewünschten Tabelle. Hierbei werden sowohl die Spaltennamen als auch sämtliche Datensätze der ausgewählten Tabelle aus der Datenbank geladen und an das Hauptprogramm zurückgegeben.

Die Funktion `darstellung_tabelle()` erstellt anschließend das grafische Fenster der Anwendung. Dabei werden verschiedene Eigenschaften wie der Fenstertitel und die Fenstergröße festgelegt.

Für die tabellarische Darstellung der Daten wird das Tkinter-Element Treeview verwendet. Die zuvor aus der Datenbank geladenen Spaltennamen werden automatisch als Tabellenüberschriften übernommen. Anschließend werden alle Datensätze zeilenweise in die Tabelle eingefügt.

Da größere Datenmengen nicht vollständig auf dem Bildschirm dargestellt werden können, wird zusätzlich eine vertikale Scrollbar eingebunden. Dadurch kann der Benutzer bequem durch die Datensätze navigieren.

Abschließend wird mit `mainloop()` die grafische Benutzeroberfläche gestartet. Das Programm bleibt dadurch aktiv und reagiert auf Benutzeraktionen, bis das Fenster geschlossen wird.

Dieser Programmteil bildet die Grundlage für eine benutzerfreundliche Datenverwaltung, da die Informationen aus der Datenbank übersichtlich in einer grafischen Oberfläche dargestellt werden können.

```
##### Grafisches Interface mit Tkinter #####
import tkinter as tk
from tkinter import ttk
import mariadb

# Verbindung zur Datenbank
try:
    db = mariadb.connect(
        host="10.145.240.127",
        user="root",
        password="123",
        database="Heimer.IT"
    )
except mariadb.Error as e:
    print(f"Fehler: {e}")
    sys.exit(1)

# Daten aus der Datenbank auslesen
def read_from_database(table_name):
    """Liest Daten aus einer Tabelle der Datenbank aus"""

    Parameter:
        table_name: Name der Tabelle

    Rückgabe:
        Tuple mit (Spaltennamen, Datenzeilen)

    try:
        cursor = db.cursor()
        cursor.execute(f"SELECT * FROM {table_name}")

        # Hole Spaltennamen
        column_names = [desc[0] for desc in cursor.description]

        # Hole alle Daten
        rows = cursor.fetchall()

        cursor.close()

        return column_names, rows
    except mariadb.Error as e:
        print(f"Fehler: {e}")
        return [], []

# Tkinter-Setup
def darstellung_tabelle():
    """Erstellt und zeigt das Tkinter-Fenster mit einer Tabelle an"""

    # Erstelle Hauptfenster
    root = tk.Tk()
    root.title("Heimer.IT-Systeme - Datenverwaltung")
    root.geometry("1000x600")

    # Lese Daten aus der artikel_Tabelle
    column_names, rows = read_from_database('artikel')

    # Erstelle Treeview (Tabelle)
    tree = ttk.Treeview(root, columns=column_names, show="headings")

    # Definiere Spalten
    for col in column_names:
        tree.heading(col, text=col)  # Spaltenüberschrift
        tree.column(col, width=100)  # Spaltenbreite

    # Füge Daten in die Tabelle ein
    for row in rows:
        tree.insert("", "end", values=row)

    # Scrollbar hinzufügen
    scrollbar = ttk.Scrollbar(root, orient="vertical", command=tree.yview)
    tree.config(yscrollcommand=scrollbar.set)

    # Platziere Tabelle und Scrollbar
    scrollbar.pack(side="right", fill="y")
    tree.pack(side="left", fill="both", padx=10, pady=10)

    # Starte das Fenster
    root.mainloop()

# Aufruf der GUI
darstellung_tabelle()
```



```
(.venv) → lf8 git:(main) × ls
01_ConnectionTest.py 2.1.py 2.2.py codes CSVExport.py CSVtoXML.py GUITabelle.py lagerbestand.csv login
.py printSQL.py README.MD v7dashboard.py XMLtoHTML.py
(.venv) → lf8 git:(main) × python CSVtoXML.py
✓ XML-Konvertierung erfolgreich: lagerbestand.xml
(.venv) → lf8 git:(main) × ls
01_ConnectionTest.py 2.2.py CSVExport.py GUITabelle.py lagerbestand.xml main.py README.MD
2.1.py codes CSVtoXML.py lagerbestand.csv login_log.txt printSQL.py v7dashboard.py
(.venv) → lf8 git:(main) × python -c "$($(slurp))" -l wsl -c py
```

1.5 KONVERTIERUNG EINER CSV-DATEI IN EIN XML-DOKUMENT

Dieser Programmabschnitt dient der Umwandlung von Daten aus einer CSV-Datei in das XML-Format. XML (Extensible Markup Language) wird häufig für den Datenaustausch zwischen verschiedenen Anwendungen und Systemen verwendet, da die Daten hierarchisch und strukturiert dargestellt werden können.

Die Funktion `csv_to_xml()` erhält als Parameter den Namen der Eingabedatei (CSV) sowie den Namen der Ausgabedatei (XML). Zunächst wird ein Root-Element mit dem Namen Datenbank erstellt, das als oberstes Element des XML-Dokuments dient.

Anschließend wird die CSV-Datei geöffnet und mit Hilfe des DictReader eingelesen. Der Vorteil dieser Methode besteht darin, dass die Spaltenüberschriften der CSV-Datei automatisch als Schlüssel verwendet werden. Dadurch können die Daten einfacher verarbeitet werden.

Für jede Zeile der CSV-Datei wird ein neues XML-Element mit dem Namen Datensatz erzeugt. Innerhalb dieses Elements werden alle Felder der jeweiligen Zeile als Unterelemente angelegt. Die Spaltennamen werden dabei als XML-Tags verwendet und die zugehörigen Werte als Inhalt der Elemente gespeichert.

Nachdem alle Datensätze verarbeitet wurden, wird das XML-Dokument formatiert und übersichtlich eingerückt. Dies verbessert die Lesbarkeit der erzeugten Datei erheblich.

Zum Abschluss wird die fertige XML-Struktur in eine neue XML-Datei geschrieben. Im vorliegenden Beispiel wird die zuvor exportierte Datei `lagerbestand.csv` eingelesen und als `lagerbestand.xml` gespeichert.

Durch diese Konvertierung können die Daten nicht nur in Tabellenform, sondern auch in einem standardisierten XML-Format weiterverwendet werden. Dies erleichtert den Datenaustausch mit anderen Anwendungen und Systemen.

CODE

```
import csv
import xml.etree.ElementTree as ET
import xml.dom.minidom as minidom

def csv_to_xml(input_file, output_file):
    """
    Konvertiert eine CSV-Datei in ein XML-Dokument

    Parameter:
        input_file: Name der CSV-Eingabedatei
        output_file: Name der XML-Ausgabedatei
    """
    try:
        # Erstelle Root-Element
        root = ET.Element('Datenbank')

        # Öffne und lese CSV-Datei
        with open(input_file, 'r', encoding='utf-8') as csvfile:
            csv_reader = csv.DictReader(csvfile, delimiter=',') # Lese mit
            # Spaltennamen

            # Verarbeite jede Zeile
            for row in csv_reader:
                # Erstelle Element für jede Zeile
                item = ET.SubElement(root, 'Datensatz')

                # Füge jedes Feld als Unterelement hinzu
                for key, value in row.items():
                    field = ET.SubElement(item, key)
                    field.text = str(value) if value else ""

            # Formatiere und schreibe XML
            xml_str =
            minidom.parseString(ET.tostring(root)).toprettyxml(indent=" ")

            # Entferne die erste XML-Deklarationszeile (wird doppelt erzeugt)
            xml_str = '\n'.join(xml_str.split('\n')[1:])

            with open(output_file, 'w', encoding='utf-8') as xmlfile:
                xmlfile.write(xml_str)

            print(f"✓ XML-Konvertierung erfolgreich: {output_file}")

    except FileNotFoundError as e:
        print(f"✗ Fehler: Datei nicht gefunden - {e}")
    except Exception as e:
        print(f"✗ Fehler bei der Konvertierung: {e}")

# Beispiel: CSV-Datei in XML konvertieren
csv_to_xml("lagerbestand.csv", "lagerbestand.xml")
```

1.6 Darstellung von XML-Daten als HTML-Webseite

```
(.venv) → lf8 git:(main) × ls
01_ConnectionTest.py  2.2.py  CSVExport.py  GUITabelle.py  lagerbestand.xml  main.py  README.MD  XMLtoHTML.py
2.1.py               codes  CSVtoXML.py  lagerbestand.csv  login_log.txt  printSQL.py  v7dashboard.py
(.venv) → lf8 git:(main) × python XMLtoHTML.py
✓ HTML-Export erfolgreich: lagerbestand.html
(.venv) → lf8 git:(main) × ls
01_ConnectionTest.py  2.2.py  CSVExport.py  GUITabelle.py  lagerbestand.html  login_log.txt  printSQL.py  v7dashboard.py
2.1.py               codes  CSVtoXML.py  lagerbestand.csv  lagerbestand.xml  main.py  README.MD  XMLtoHTML.py
(.venv) → lf8 git:(main) × grim -g "$(slurp)" - | wl-copy
```

Dieser Programmabschnitt dient dazu, die zuvor erzeugte XML-Datei in eine benutzerfreundliche HTML-Webseite umzuwandeln. Dadurch können die Daten direkt in einem Webbrowser angezeigt werden, ohne dass spezielle Programme oder Kenntnisse über XML erforderlich sind.

Zunächst wird die XML-Datei mit Hilfe der Bibliothek `xml.etree.ElementTree` eingelesen und analysiert. Das Programm lädt die gesamte XML-Struktur und greift auf die enthaltenen Datensätze zu.

Anschließend werden die Spaltennamen automatisch aus dem ersten Datensatz der XML-Datei ermittelt. Dadurch kann die HTML-Tabelle dynamisch erstellt werden, unabhängig davon, welche Datenstruktur die XML-Datei besitzt.

Im nächsten Schritt wird der HTML-Code erzeugt. Dabei werden die grundlegenden Bestandteile einer Webseite erstellt, darunter der Dokumentkopf, die Formatierung mit CSS sowie die eigentliche Tabelle zur Darstellung der Daten.

Die Spaltenüberschriften werden automatisch aus den XML-Tags generiert. Danach werden alle Datensätze aus der XML-Datei ausgelesen und zeilenweise in die HTML-Tabelle eingefügt. Jedes XML-Feld wird dabei als Tabellenzelle dargestellt.

Zusätzlich enthält die Webseite ein CSS-Layout, welches für eine übersichtliche und moderne Darstellung sorgt. Dazu gehören unter anderem farbige Tabellenüberschriften, Zeilenhervorhebungen beim Überfahren mit der Maus sowie eine verbesserte Lesbarkeit durch wechselnde Hintergrundfarben.

Nach der Erstellung wird die fertige HTML-Datei gespeichert und kann anschließend direkt in einem Webbrowser geöffnet werden. Im vorliegenden Beispiel wird die Datei `lagerbestand.xml` eingelesen und als `lagerbestand.html` ausgegeben.

Dieser Programmteil bildet den letzten Schritt der Datenaufbereitung. Die ursprünglich in der Datenbank gespeicherten Informationen werden über mehrere Verarbeitungsschritte bis hin zu einer fertigen Webseite transformiert und für Endbenutzer leicht zugänglich gemacht.

CODE

```
import xml.etree.ElementTree as ET
def xml_to_html(xml_file, html_output_file):
    """
    Konvertiert eine XML-Datei in ein HTML-Dokument und stellt es tabellarisch
    dar.
    """
    # Parameter
    xml_file = Name der XML-Eingabedatei
    html_output_file = Name der HTML-Ausgabedatei

    # XML-Daten in einem Web-Browser visual darstellen
    # - Ermöglicht einfaches Drucken und Weitergabe der Daten
    # - Ermöglicht einfaches Drucken und Weitergabe der Daten

    # Parse XML-Datei
    tree = ET.parse(xml_file)
    root = tree.getroot()

    # Ermittle Spaltennamen aus dem ersten Datensatz
    if root.tag != 'records':
        raise ValueError("Die XML-Datei muss einen Root-Tag mit dem Namen 'records' enthalten.")
    columns = [child.tag for child in root.getchildren()]

    # Erstelle HTML-Struktur
    html_content = """
<DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>XML-Export</title>
<style>
<!-- CSS-Code -->
</style>
</head>
<body>
<table>
<thead>
<tr>
<th>{<!-- Spaltenname -->}</th>
</tr>
</thead>
<tbody>
<tr>
<td>{<!-- Daten -->}</td>
</tr>
</tbody>
</table>
</body>
</html>
"""

    # Füge Spaltenkopie hinzu
    for col in columns:
        html_content = f"""
        <th>{col}</th>>"""
        html_content += """
        </tr>
        </thead>
        <tbody>
        <tr>
        <td>{<!-- Daten -->}</td>
        </tr>
        </tbody>
        </table>
        </body>
        </html>
"""

    # Füge Datensätze hinzu
    for record in root.findall('record'):
        for col in columns:
            text = record.find(col)
            if text is not None and text.text is not None:
                html_content += f"""
                <tr>
                <td>{text.text}</td>
                </tr>
                </tbody>
                </table>
                </body>
                </html>
"""

    # Erstelle HTML-Datei
    with open(html_output_file, 'w', encoding='utf-8') as f:
        f.write(html_content)

    # Aufruf der Funktion
    xml_to_html('lagerbestand.xml', 'lagerbestand.html')
```


DOKUMENTATION 1.1

Als Übersicht: Projekt: Verwaltung und Export der Datenbank in verschiedene Formate Ziel: Bereitstellung von Daten in XML, CSV und HTML für verschiedene Abteilungen (Read Write Regeln + Bonus => KundenVerwaltung Web-GUI)

MEILENSTEIN-ÜBERSICHT UND ZEITAUFWAND

	Nummer	Meilenstein	Geplant	Actual
	1.0	Quellcode-Dokumentation	2h	2h
	1.1	Datenbankverbindung mit MariaDB	2h	0,5h
	1.2	SQL-Abfrage mit Print-Ausgabe	1h	0,5h
	1.3	CSV-Export aus Datenbank	2h	1h
	1.4	GUI mit Tkinter (Tabellendarstellung)	2h	2h
	1.5	CSV zu XML Konvertierung	1h	0,5h
	1.6	Erweiterung: XML zu HTML	1h	1h
	1.7	Dokumentation	1h	0,5h
	GESAMT		12h	6h

	Nummer	Meilenstein	Geplant	Actual
	2.1	Dropdown-Mitarbeiterauswahl-Passwort	2h	1h
	2.2	Verfügbarkeit der Tools	2h	1h
	2.3	Überraschung-Funktionalität(Verwaltung GUI)	2h	3h
	GESAMT		6h	5h

DOKUMENTATION 1.2

AUFGETRETENE PROBLEME UND LÖSUNGEN

1. Verbindung der DB (1.1) ————— Hürde: Server-IP und Passwort korrekt eingeben Lösung: Überprüfung der Credentials (10.145.240.127, root, 123, Heiner_IT)
2. SPALTENNAMEN (1.2) ————— Problem: Spaltenköpfe nicht dynamisch ausgelesen Hürde: cursor.description verwenden für Metadaten Lösung: [desc[0] for desc in cursor.description]
3. CSV Trennzeichen (1.3) ————— Problem: CSV-Datei mit Kommas statt Semikolo Hürde: Deutsche Excel-Kompatibilität erfordert Semikolon Lösung: delimiter=';' in csv.writer() (**AUFGETRETENE PROBLEME UND LÖSUNGEN**)
4. Verbindung der DB (1.1) ————— Hürde: Server-IP und Passwort korrekt eingeben Lösung: Überprüfung der Credentials (10.145.240.127, root, 123, Heiner_IT)
5. SPALTENNAMEN (1.2) ————— Problem: Spaltenköpfe nicht dynamisch ausgelesen Hürde: cursor.description verwenden für Metadaten Lösung: [desc[0] for desc in cursor.description]
6. CSV Trennzeichen (1.3) ————— Problem: CSV-Datei mit Kommas statt Semikolo Hürde: Deutsche Excel-Kompatibilität erfordert Semikolon Lösung: delimiter=';' in csv.writer() (Als Übersicht: Projekt: Verwaltung und Export der Datenbank in verschiedene Formate Ziel: Bereitstellung von Daten in XML, CSV und HTML für verschiedene Abteilungen (Read Write Regeln + Bonus => KundenVerwaltung Web-GUI))
7. MEILENSTEIN-ÜBERSICHT UND ZEITAUFWAND

Nummer	Meilenstein			Geplant	Actual	
	Quellcode-Dokumentation			2h	2h	1.0
	Datenbankverbindung	mit	MariaDB	2h	0,5h	1.1
	SQL-Abfrage	mit	Print-Ausgabe	1h	0,5h	1.2
	CSV-Export	aus	Datenbank	2h	1h	1.3
	GUI	mit	Tkinter (Tabellendarstellung)	2h	2h	1.4
	CSV	zu	XML Konvertierung	1h	0,5h	1.5
	Erweiterung:	XML	zu	HTML	1h	1.6
	Dokumentation			1h	0,5h	1.7
GESAMT				12h	6h	

Nummer	Meilenstein			Geplant	Actual	
	Dropdown-Mitarbeiterauswahl-Passwort			2h	1h	2.1
	Verfügbarkeit	der	Tools	2h	1h	2.2
	Überraschung-Funktionalität(Verwaltung GUI)			2h	3h	2.3
GESAMT				6h	5h	

- 10.)
11. tkinterr GUI (1.4) ————— Problem: Scrollbar nicht funktionierend hürde: Korrekte Platzierung und Konfiguration lösung: Scrollbar mit tree.configure(yscroll=scrollbar.set) verbinden
12. XML formatierung (1.5) ————— Problem: XML nicht korrekt eingerückt (pretty_print) Hürde: minidom.parseString() nutzen für Formatierung Lösung: XML mit toprettyxml(indent=" ") formatieren
13. HTML (1.6) ————— problem: HTML-Tabelle nicht ansehnlich Hürde: CSS-Styling hinzufügen Lösung: Inline-CSS mit Hover-Effekten und Zebra-Stripes
- 14.)
15. tkinterr GUI (1.4) ————— Problem: Scrollbar nicht funktionierend hürde: Korrekte Platzierung und Konfiguration lösung: Scrollbar mit tree.configure(yscroll=scrollbar.set) verbinden
16. XML formatierung (1.5) ————— Problem: XML nicht korrekt eingerückt (pretty print) Hürde: minidom.parseString() nutzen für Formatierung Lösung: XML mit toprettyxml(indent=" ") formatieren
17. HTML (1.6) ————— problem: HTML-Tabelle nicht ansehnlich Hürde: CSS-Styling hinzufügen Lösung: Inline-CSS mit Hover-Effekten und Zebra-Stripes

DOKUMENTATION 1.3

VERWENDETE BIBLIOTHEKEN:

• mariadb: Datenbankverbindung und -abfragen • csv: CSV-Export und -Verarbeitung • xml.etree.ElementTree: XML-Verarbeitung • xml.dom.minidom: XML-Formatierung (Pretty Print) • tkinter: Grafische Benutzeroberfläche • sys: Systemfunktionen (Fehlerbehandlung) • flask: Web-GUI für 2.3 • datetime: Datums- und Zeitfunktionen • hashlib: Passwort-Hashing und Sicherheitsfunktionen

SERVER-KONFIGURATION als DOKU:

Host: 10.145.240.127 User: root Passwort: 123 Datenbank: Heiner_IT Typ: MariaDB

DATEIEN UND OUTPUTS ALS DOKU:

Generierte Dateien: 01_ConnectionTest.py - Verbindungstest 1.1 printSQL.py - SQL-Ausgabe 1.2 CSVExport.py - CSV-Export 1.3 GUITabelle.py - Tkinter GUI 1.4 CSVtoXML.py - CSV zu XML Konvertierung 1.5 XMLtoHTML.py - XML zu HTML Konvertierung 1.6

Nummer	Meilenstein	Geplant	Actual
1.0	Quellcode-Dokumentation	2h	2h
1.1	Datenbankverbindung mit MariaDB	2h	0,5h
1.2	SQL-Abfrage mit Print-Ausgabe	1h	0,5h
1.3	CSV-Export aus Datenbank	2h	1h
1.4	GUI mit Tkinter (Tabellendarstellung)	2h	2h
1.5	CSV zu XML Konvertierung	1h	0,5h
1.6	Erweiterung: XML zu HTML	1h	1h
1.7	Dokumentation	1h	0,5h
GESAMT		12h	6h

Nummer	Meilenstein	Geplant	Actual
2.1	Dropdown-Mitarbeiterauswahl-Passwort	2h	1h
2.2	Verfügbarkeit der Tools	2h	1h
2.3	Überraschung-Funktionalität(Verwaltung GUI)	2h	3h
GESAMT		6h	5h

Heiner IT-Systems GmbH

Willkommen, Lager!

Abteilung: Lager

Passwort: *****

OK

Heiner IT-Systems GmbH

Mitarbeiter-Verwaltungssystem

Abteilung: Lager

Passwort: *****

Heiner IT-Systems GmbH

Passwort ist falsch! schreib einfach den Abteilung Namen

Passwort: *****

OK

2.1 DROPDOWN-MITARBEITERAUSWAHL MIT PASSWORT-LOGIN

Dieser Programmabschnitt implementiert ein grafisches Login-System für die Anwendung. Ziel ist es, den Zugriff auf das Verwaltungssystem auf berechnigte Benutzergruppen zu beschränken und gleichzeitig eine einfache und benutzerfreundliche Anmeldung zu ermöglichen.

Die Klasse LoginWindow erstellt das komplette Anmeldefenster mit Hilfe der Python-Bibliothek Tkinter. Beim Start der Anwendung wird ein Fenster geöffnet, das aus mehreren grafischen Elementen besteht:

- Firmenname und Systembezeichnung
- Dropdown-Menü zur Auswahl der Abteilung
- Passwort-Eingabefeld
- Login-Button
- Abbrechen-Button

Das Passwortfeld verwendet eine Maskierung mit Sternsymbolen, sodass die Eingabe für andere Personen nicht sichtbar ist.

Wird der Login-Button betätigt oder die Eingabetaste gedrückt, wird die Methode login_clicked() ausgeführt. Diese überprüft zunächst, ob eine Abteilung ausgewählt wurde und ob ein Passwort eingegeben wurde. Anschließend erfolgt die Passwortvalidierung.

Im Rahmen dieses Demonstrationsprojekts entspricht das Passwort dem Namen der ausgewählten Abteilung. Dadurch kann die Funktionsweise eines Login-Prozesses vereinfacht dargestellt werden. Wird ein falsches Passwort eingegeben, erscheint eine Fehlermeldung und das Passwortfeld wird automatisch geleert.

Bei erfolgreicher Anmeldung wird die ausgewählte Abteilung in einer globalen Variable gespeichert. Anschließend erhält der Benutzer eine Willkommensmeldung und das Login-Fenster wird geschlossen. Die Anwendung kann danach mit den Informationen des angemeldeten Benutzers weiterarbeiten.

Dieser Programmteil bildet die Grundlage der Benutzeranmeldung und stellt sicher, dass Benutzer zunächst authentifiziert werden, bevor sie auf die Funktionen des Systems zugreifen können.

CODE

```
##### Dropdown-Mitarbeiterauswahl mit Passwort-Login 21 #####
import tkinter as tk
from tkinter import ttk
from tkinter import messagebox

# Definiere Abteilungen (Mitarbeitergruppen) - USER DICTIONARY
mitarbeiter_gruppen = {
    "Lager": "Lager",
    "Verwaltung": "Verwaltung",
    "Marketing": "Marketing",
    "Geschäftsführung": "Geschäftsführung"
}

# Globale Variable für aktuell angemeldete Abteilung - immer wird gezeigt wenn man den ding startet
aktuelle_abteilung = None
# Wenn app ist einfach Glas und da haben wir schon alles als funktionen definiert
class LoginWindow:
    """Klasse für das Login-Fenster mit Dropdown-Auswahl
    Ermöglicht Auswahl der Abteilung und Passwortvalidierung"""

    def __init__(self, root):
        """Initialisiert das Login-Fenster"""
        # root: Das Tkinter-Hauptfenster
        self.root = root
        self.root.title("Heiner IT-Systems GmbH - Login")
        self.root.geometry("400x300")
        self.root.resizable(False, False)
        self.root.configure(bg="#f0f0f0")

        # Zentriere Fenster auf dem Bildschirm
        self.center_window()

        # Erstelle GUI-Elemente
        self.create_widgets()

    def center_window(self):
        """Zentriert das Fenster auf dem Bildschirm"""
        self.root.update_idletasks()
        width = self.root.winfo_width()
        height = self.root.winfo_height()
        x = (self.root.winfo_screenwidth() // 2) - (width // 2)
        y = (self.root.winfo_screenheight() // 2) - (height // 2)
        self.root.geometry(f"{width}x{height}+{x}+{y}")

    def create_widgets(self):
        """Erstellt alle GUI-Elemente für das Login-Fenster"""

        # Titel
        title_label = tk.Label(
            text="Heiner IT-Systems GmbH",
            font=("Arial", 16, "bold"),
            fg="black"
        )
        title_label.pack(pady=10)

        # Unterlabel
        subtitle_label = tk.Label(
            text="Mitarbeiter-Verwaltungssystem",
            font=("Arial", 12),
            fg="gray"
        )
        subtitle_label.pack(pady=5)

        # Frame für Login-Elemente
        login_frame = tk.Frame(self.root, bg="white", border=1)
        login_frame.pack(pady=10, padx=10, fill="both", expand=True)

        # Abteilungs-Label
        dept_label = tk.Label(
            text="Abteilung:",
            font=("Arial", 12),
            fg="black"
        )
        dept_label.pack(pady=5)

        # Abteilungs-Dropdown
        dept_dropdown = tk.OptionMenu(
            login_frame,
            self.dept_var,
            *mitarbeiter_gruppen.keys()
        )
        dept_dropdown.pack(pady=5)

        # Passwort-Label
        pass_label = tk.Label(
            text="Passwort:",
            font=("Arial", 12),
            fg="black"
        )
        pass_label.pack(pady=5)

        # Passwort-Eingabefeld
        pass_entry = tk.Entry(
            login_frame,
            show="*",
            font=("Arial", 12)
        )
        pass_entry.pack(pady=5)

        # Login-Button
        login_button = tk.Button(
            login_frame,
            text="Login",
            command=self.login_clicked,
            font=("Arial", 12, "bold"),
            bg="blue",
            fg="white"
        )
        login_button.pack(pady=10)

        # Abbrechen-Button
        cancel_button = tk.Button(
            login_frame,
            text="Abbrechen",
            command=self.root.quit,
            font=("Arial", 12),
            bg="gray",
            fg="black"
        )
        cancel_button.pack(pady=10)

    def login_clicked(self):
        """Validiert das Login"""
        # Validierung: Abteilung ausgewählt?
        if self.dept_var.get() == "":
            messagebox.showerror("Fehler", "Bitte wählen Sie eine Abteilung")
            return

        # Validierung: Passwort eingegeben?
        if pass_entry.get() == "":
            messagebox.showerror("Fehler", "Bitte geben Sie das Passwort ein")
            return

        # Validierung: Passwort korrekt?
        if self.dept_var.get() != pass_entry.get():
            messagebox.showerror("Fehler", "Passwort ist falsch! schreib einfach den Abteilung Namen")
            return

        # Login erfolgreich
        self.aktuelle_abteilung = self.dept_var.get()
        self.root.destroy()

        # Starte das Login-Fenster
        self.root.mainloop()

# Hauptfunktion
def main():
    """Startet das Login-Fenster"""
    root = tk.Tk()
    login_window = LoginWindow(root)
    root.mainloop()

if __name__ == "__main__":
    main()
```

2.2 Hauptprogramm mit rollenbasiertem Zugriff auf Werkzeuge

Willkommen, Geschäftsführung!

Verfügbare Tools:

Tool auswählen:

DBAusgabeFenster
print_SQL_Ausgabe
DBinCSV
DBAusgabeFenster
CSV to XML

Tool-Beschreibung:

Zeigt Datenbankdaten in einem grafischen Fenster
Funktion: Tabellarische Darstellung mit Scrollbar
Output: GUI-Fenster mit interaktiver Tabelle
Sortierung: Durch Klick auf Spaltenköpfe möglich

Willkommen, Lager!

Verfügbare Tools:

Tool auswählen:

-- Tool wählen --
print_SQL_Ausgabe
DBAusgabeFenster

Tool-Beschreibung:

Dieser Programmabschnitt bildet das zentrale Hauptprogramm der Anwendung. Nach erfolgreicher Anmeldung erhält der Benutzer Zugriff auf ein Hauptfenster, in dem die für seine Abteilung freigegebenen Werkzeuge (Tools) angezeigt werden.

Zu Beginn werden die verfügbaren Werkzeuge für jede Abteilung in einer Konfigurationsstruktur definiert. Dadurch wird festgelegt, welche Funktionen den einzelnen Benutzergruppen zur Verfügung stehen. Beispielsweise kann die Geschäftsführung auf alle Werkzeuge zugreifen, während andere Abteilungen nur die für ihre Aufgaben relevanten Funktionen verwenden können.

Nach dem Login wird das Hauptfenster der Anwendung geöffnet. Dieses zeigt den Namen der angemeldeten Abteilung sowie die verfügbaren Werkzeuge an. Die Oberfläche wurde mit Tkinter erstellt und bietet eine benutzerfreundliche Bedienung über Dropdown-Menüs und Schaltflächen.

Die verfügbaren Werkzeuge werden dynamisch aus der Konfiguration geladen. Dadurch passt sich die Oberfläche automatisch an die jeweilige Benutzerrolle an. Benutzer sehen ausschließlich die Funktionen, für die sie berechtigt sind.

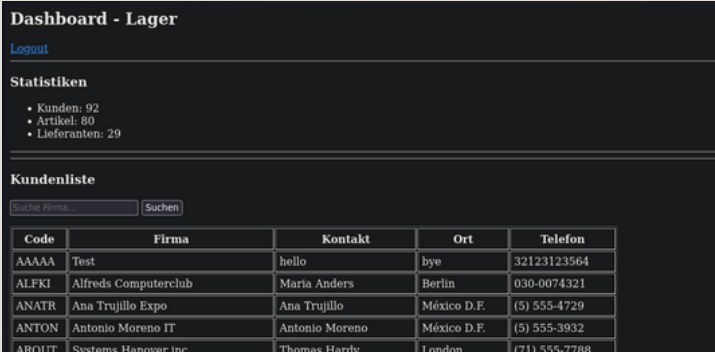
Zusätzlich enthält die Anwendung eine Beschreibung für jedes Werkzeug. Sobald ein Tool ausgewählt wird, erscheint eine Erklärung im unteren Bereich des Fensters. Dadurch erhält der Benutzer Informationen über Zweck, Funktion und Ausgabe des ausgewählten Programms.

Wird die Schaltfläche „Ausführen“ betätigt, startet das Programm die zugehörige Python-Datei mit Hilfe des Moduls subprocess. Dadurch können einzelne Funktionen als eigenständige Programme ausgeführt werden, ohne dass das Hauptprogramm beendet werden muss.

Darüber hinaus verfügt die Anwendung über eine Logout-Funktion. Nach der Abmeldung wird das Hauptfenster geschlossen und das Login-Fenster erneut geöffnet, sodass sich ein anderer Benutzer anmelden kann.

Durch diese Umsetzung wurde ein einfaches rollenbasiertes Berechtigungssystem realisiert. Die Anwendung stellt sicher, dass verschiedene Abteilungen nur auf die für sie vorgesehenen Funktionen zugreifen können. Dadurch wird die Bedienung vereinfacht und gleichzeitig die Übersichtlichkeit des Systems erhöht.





Zu Beginn wird eine Flask-Anwendung erstellt und mit einem geheimen Sitzungsschlüssel (Secret Key) konfiguriert. Dieser Schlüssel wird verwendet, um Benutzersitzungen sicher zu verwalten und angemeldete Benutzer während ihrer Nutzung der Anwendung zu identifizieren.

Anschließend wird eine Verbindung zur MariaDB-Datenbank aufgebaut. Über diese Verbindung können Daten aus den verschiedenen Tabellen gelesen, hinzugefügt, bearbeitet oder gelöscht werden.

Für die Benutzeranmeldung wurde ein einfaches Authentifizierungssystem implementiert. Die verfügbaren Benutzergruppen werden in einer Datenstruktur hinterlegt. Die Passwörter werden dabei nicht im Klartext gespeichert, sondern mit dem SHA-256-Verfahren gehasht. Dadurch wird die Sicherheit der Anwendung erhöht.

Bei jedem Anmeldeversuch wird das eingegebene Passwort ebenfalls gehasht und mit dem gespeicherten Hashwert verglichen. Erfolgreiche und fehlgeschlagene Anmeldungen werden zusätzlich in einer Logdatei dokumentiert. Dadurch können Anmeldeaktivitäten nachvollzogen werden.

Nach erfolgreicher Anmeldung wird der Benutzer auf das Dashboard weitergeleitet. Das Dashboard stellt verschiedene Informationen aus der Datenbank bereit, darunter:

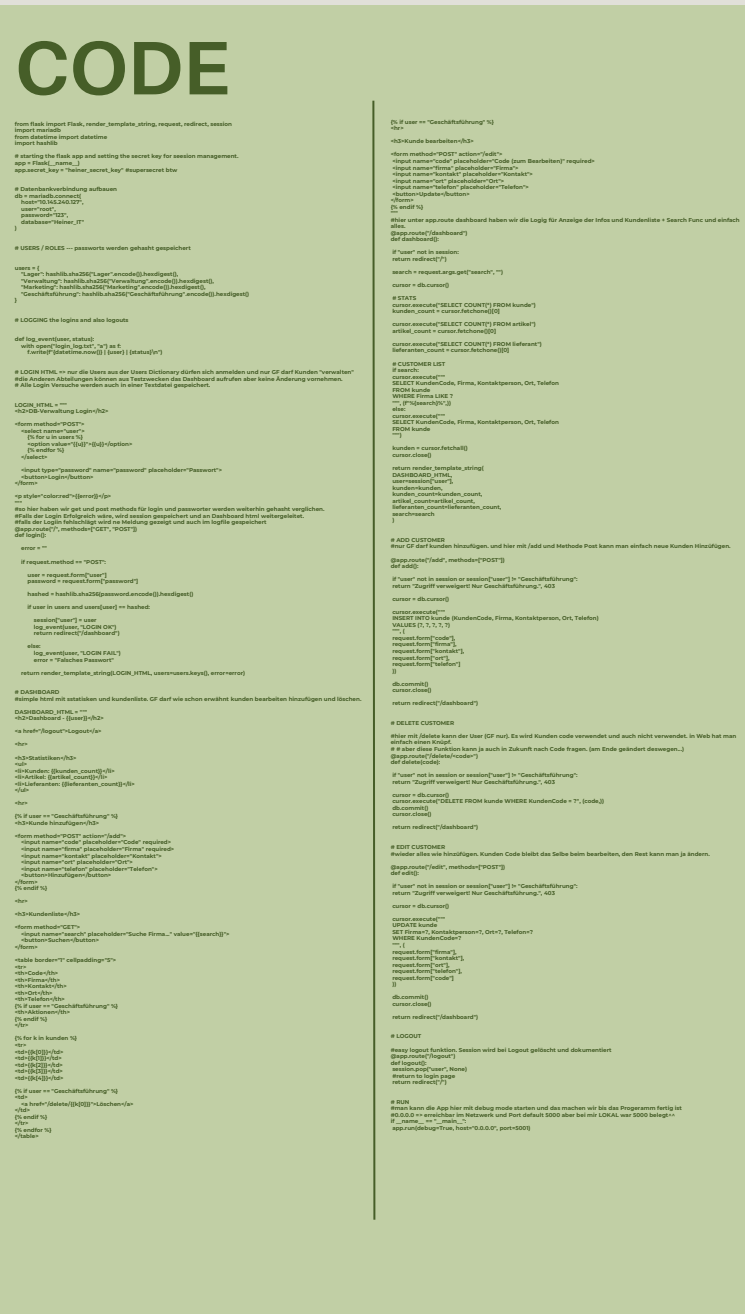
- Die Anwendung verfügt außerdem über ein rollenbasiertes Berechtigungssystem. Während alle Benutzer das Dashboard und die Kundenübersicht einsehen können, stehen Verwaltungsfunktionen ausschließlich der Geschäftsführung zur Verfügung.

Für jede dieser Aktionen existiert eine eigene Route innerhalb der Flask-Anwendung. Die Änderungen werden direkt in der MariaDB-Datenbank gespeichert und stehen unmittelbar allen Benutzern zur Verfügung.

Zusätzlich wurde eine Logout-Funktion implementiert. Beim Abmelden wird die aktive Sitzung beendet und der Benutzer zurück auf die Login-Seite geleitet.

Die Anwendung wird abschließend über einen lokalen Webserver gestartet. Dadurch kann sie über einen Webbrowser aufgerufen und innerhalb des Schulnetzwerks verwendet werden.

Dieser Programmteil verbindet die zuvor entwickelten Datenbank- und Verwaltungsfunktionen mit einer modernen Weboberfläche und stellt die zentrale Benutzeranwendung des Projekts dar.





Gruppe Mitglieder

Amirhossein Gozashti

Reza Abutalebi

Elias Könemann